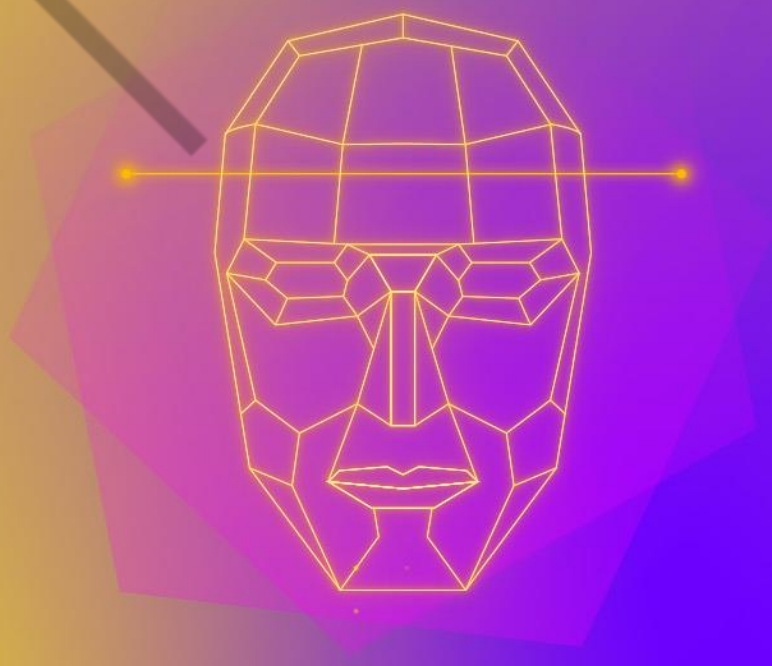
BUILDING STATISTICAL MODELS

VISÃO COMPUTACIONAL (DETEÇÃO DE FACES E OBJETOS)



10

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 - Milestone de visão computacional 5
Figura 2- Exemplo de classificação, localização e detecção de objeto 6
Figura 3 - Exemplo de detecção vs segmentação.....7
Figura 4 - Exemplo de detecção de objeto com SIFT 9
Figura 5 - Exemplo de detecção de invasão 10
Figura 6- Exemplo de detecção de objetos no trânsito 10
Figura 7- Exemplo de análise de comportamento de compra 11
Figura 8 - Exemplo de diagnóstico por imagem 12
Figura 9 - Exemplo de segmentação de objeto 13
Figura 10 - Exemplo de classificação VGG16.....22
Figura 11 - Exemplo de classificação MobileNet.....24
Figura 12– Organização dos dados 26
Figura 13 - Organização dos dados csv.....30
Figura 14 - Organização dos dados csv.....37
Figura 15 - exemplo dataset.....44
Figura 16 - exemplo treinamento dataset.....51
Figura 17 - Treinamento resultado 53

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Detecta objetos 15
Código-fonte 2 – Detecção de faces com Haar Cascade 17
Código-fonte 3 – Exemplo modelo VGG16 pré-treinada 21
Código-fonte 4 – Carrega modelo MobileNet 23
Código-fonte 5 – Carrega MobileNet 23
Código-fonte 6 – Carrega dados 25
Código-fonte 7 – Estrutura os dados csv 30
Código-fonte 8 – Função auxiliar bounding box 32
Código-fonte 9 – Função converte bounding box 33
Código-fonte 10 – Função data augmentation 36
Código-fonte 11 – Verifica dataset apõs data augmentation 37
Código-fonte 12 – Continua código 41
Código-fonte 13 – Embaralha dados 41
Código-fonte 14 – exibe resultado 43
Código-fonte 15 – Separa dados testes treino 45
Código-fonte 16 – Carrega modelo 46
Código-fonte 17 – Continua testes 48
Código-fonte 18 – compila o modelo 50
Código-fonte 19 – Compila modelo 50
Código-fonte 20 – Exibe resultado 52
Código-fonte 21 – Salva modelo 53
Código-fonte 22 – Testa resultado 56

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

1 VISÃO COMPUTACIONAL (DETECÇÃO DE FACES E OBJETOS) 5
1.1 História e Evolução da Visão Computacional.....5
1.2 Detecção de Objetos na Visão Computacional 6
1.3 Segmentação de Imagens 7
1.4 Processamento Digital de Imagens.....8
1.5 Aplicações Práticas da Visão Computacional 9

2 APLICAÇÃO DE MÉTODOS CLÁSSICOS DE DETECÇÃO DE FACES E OBJETOS 13
2.1 Detecção de Objetos por meio de detecção de contornos.....13
2.2 Detecção de Faces com Haar Cascade.....15

3 APLICAÇÃO DE APRENDIZADO PROFUNDO PARA DETECÇÃO DE FACES E OBJETOS 18
3.1 Uso de Redes Pré-treinadas para Classificação.....18
3.2 Alterando o Modelo da rede Pré-treinada para Classificação 22
3.3 Adaptando Redes de Classificação para Detecção de Objetos 24
3.4 Do Classificador ao Detector.....24

4 HANDS ON: PRATICANDO O USO DE TENSORFLOW 56
4.1 Aquecimento 56
4.2 Seu desafio 57
4.3 Dicas de solução.....57

REFERÊNCIAS 60

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1 VISÃO COMPUTACIONAL (DETECÇÃO DE FACES E OBJETOS)

A visão computacional é um campo dinâmico da ciência da computação e tecnologia que permite aos computadores interpretar e entender o mundo visual de maneira análoga à visão humana. Esse campo utiliza o desenvolvimento avançado de algoritmos que permitem extrair informações significativas de imagens digitais ou vídeos, para realizar tarefas como identificação de objetos, reconhecimento de padrões e inferências com base nos dados visuais.

1.1 História e Evolução da Visão Computacional

Desde os primeiros estudos em 1959, que se inspiravam na visão animal, até os avanços recentes com redes neurais convolucionais, a visão computacional tem evoluído significativamente. O lançamento do conjunto de dados ImageNet, em 2010, e a subsequente inovação da arquitetura AlexNet marcaram um ponto de virada, demonstrando a capacidade das redes profundas em tarefas complexas de classificação e reconhecimento de imagens, que hoje formam a base de muitas aplicações modernas de visão computacional. A Figura 1 exibe um milestone com alguns destaques da área de visão computacional.

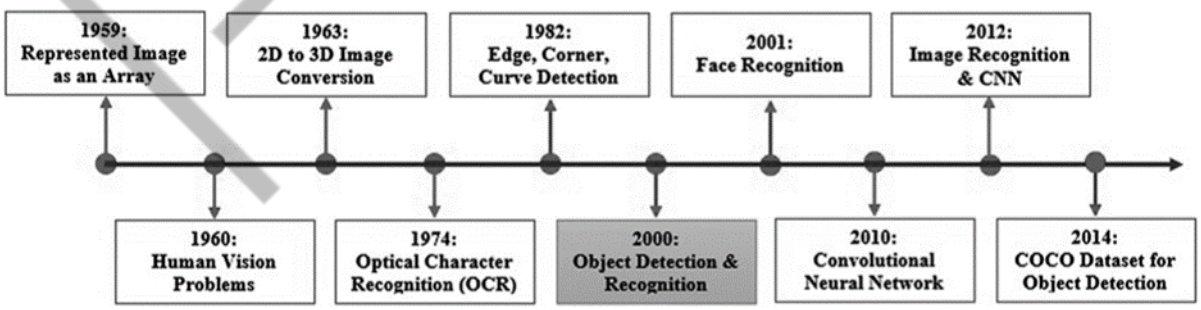


Figura 1 - Milestone de visão computacional
Fonte: Bhowmik, M. K. (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.2 Detecção de Objetos na Visão Computacional

A detecção de objetos é uma das várias tarefas dentro da visão computacional, ela compreende a identificação e localização de objetos de interesse dentro de imagens ou vídeos. Essa tarefa é geralmente realizada em duas fases principais:

- **Classificação de Objetos:** atribuição de uma categoria (como 'carro', ‘pessoa’ ou ‘árvore’) a cada objeto identificado;
- **Localização de Objetos:** utilização de caixas delimitadoras (bounding boxes) para determinar a posição e o tamanho de cada objeto em uma imagem.

Dica: ao contrário de tópicos mais complexos como detecção de objetos e segmentação de imagens, que precisam localizar (ou fornecer posições) para as características que detectam, a classificação de imagens lida com o processamento da imagem inteira como um todo e atribui um rótulo específico a ela**.**

Métodos modernos, como as arquiteturas de redes neurais YOLO (You Only Look Once), SSD (Single Shot Multibox Detector) e as variantes de R-CNN (Region-based Convolutional Neural Networks), utilizam abordagens de aprendizado profundo com redes neurais convolucionais para melhorar a precisão e a eficiência nessas tarefas. A Figura 2 demonstra as tarefas para realizar a detecção de objetos, à esquerda com a tarefa de classificação, ao centro com a adição da tarefa de localização e à direita com o resultado da detecção de objeto.

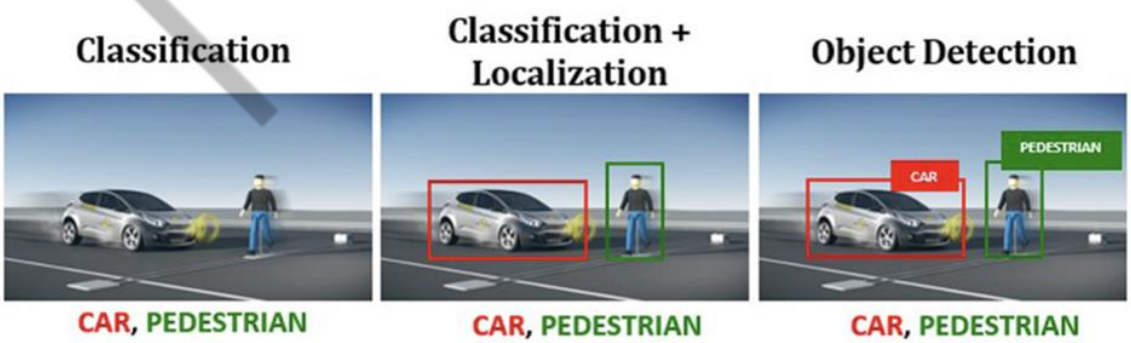


Figura 2- Exemplo de classificação, localização e detecção de objeto
Fonte: Bhowmik, M. K. (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.3 Segmentação de Imagens

A segmentação de imagens é o processo de dividir uma imagem em múltiplas regiões ou segmentos para identificar e analisar diferentes partes constituintes. Um "segmento" de uma imagem representa uma classe específica de objeto que a rede neural identificou na imagem, representado por uma máscara de pixels que pode ser usada para extraí-lo. Esse domínio popular da Visão Computacional tem sido amplamente estudado tanto com o uso de algoritmos tradicionais de processamento de imagens, como algoritmos de watershed, segmentação baseada em agrupamento e segmentação baseada em contornos, quanto com o uso de arquiteturas modernas de aprendizado profundo como PSPNet, FPN, U-Net, SegNet, entre outros. A Figura 3 ilustra a diferença entre detecção de objetos e a segmentação de instância onde cada objeto detectado é segmentado pixel a pixel, definindo seu contorno com precisão.

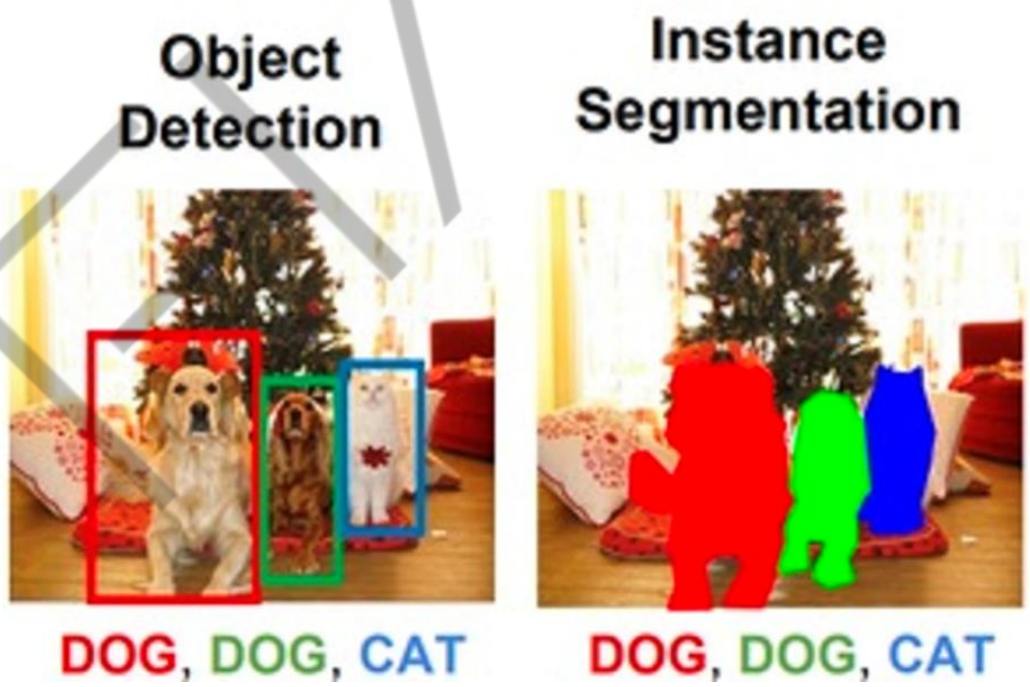


Figura 3 - Exemplo de detecção vs segmentação
Fonte: Karpathy, A., et al. (2017)

Dica: a segmentação de imagens é essencial em aplicações como diagnósticos médicos e monitoramento ambiental.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.4 Processamento Digital de Imagens

O Processamento Digital de Imagens, ou simplesmente Processamento de Imagens, é um subconjunto da Visão Computacional. Ele lida com o aprimoramento e a compreensão de imagens por meio de diversos algoritmos.

Mais do que apenas um subconjunto, o Processamento de Imagens é o precursor da visão computacional moderna, supervisionando o desenvolvimento de inúmeros algoritmos baseados em regras e otimização que levaram a visão de máquinas ao que é hoje.

O Processamento de Imagens pode ser definido como a tarefa de realizar um conjunto de operações em uma imagem com base em dados coletados por algoritmos para analisar e manipular os conteúdos de uma imagem ou os dados da imagem.

Antes do surgimento e popularização do aprendizado profundo, em 2013, quase toda a detecção de objetos era realizada por meio de técnicas clássicas de processamento de imagem e aprendizado de máquina. Técnicas comuns incluíam o método de detecção de objetos de Viola-Jones, transformadas de características invariantes à escala (SIFT) e histogramas de gradientes orientados (HOG).

A Figura 4 demonstra a aplicação da técnica SIFT para detecção de objetos, com a segmentação da capa do livro destacada em verde.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 4 - Exemplo de detecção de objeto com SIFT
Fonte: autor

Dentre outras formas, esses métodos detectaram uma série de características (features) comuns em toda a imagem e classificam seus agrupamentos usando regressão logística, histogramas de cores ou florestas aleatórias.

1.5 Aplicações Práticas da Visão Computacional

A detecção de objetos e faces é crucial para habilitar uma ampla gama de aplicações. Desde segurança até entretenimento, passando por sistemas autônomos de condução e análise de comportamento de consumidores, a habilidade de detectar e localizar objetos e rostos em tempo real é fundamental para muitos sistemas inteligentes. Neste capítulo, discutiremos como essa capacidade se integra a aplicações industriais e sociais, explorando sua relevância em diversos setores, como por exemplo:

- **Segurança e Vigilância**: uso de detecção facial para identificar indivíduos em locais públicos ou detectar atividades suspeitas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 5 - Exemplo de detecção de invasão
Fonte: https://voxel51.com/blog/how-computer-vision-is-changing-security/

- **Automotivo**: sistemas de assistência ao motorista, que detectam pedestres, sinais de trânsito e outros veículos.

Figura 6- Exemplo de detecção de objetos no trânsito
Fonte: https://www.ufrjnautilus.com/post/vis%C3%A3o-computacional-e-carros-aut%C3%B4nomos

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Varejo**: análise de fluxo de clientes e comportamento de compra.

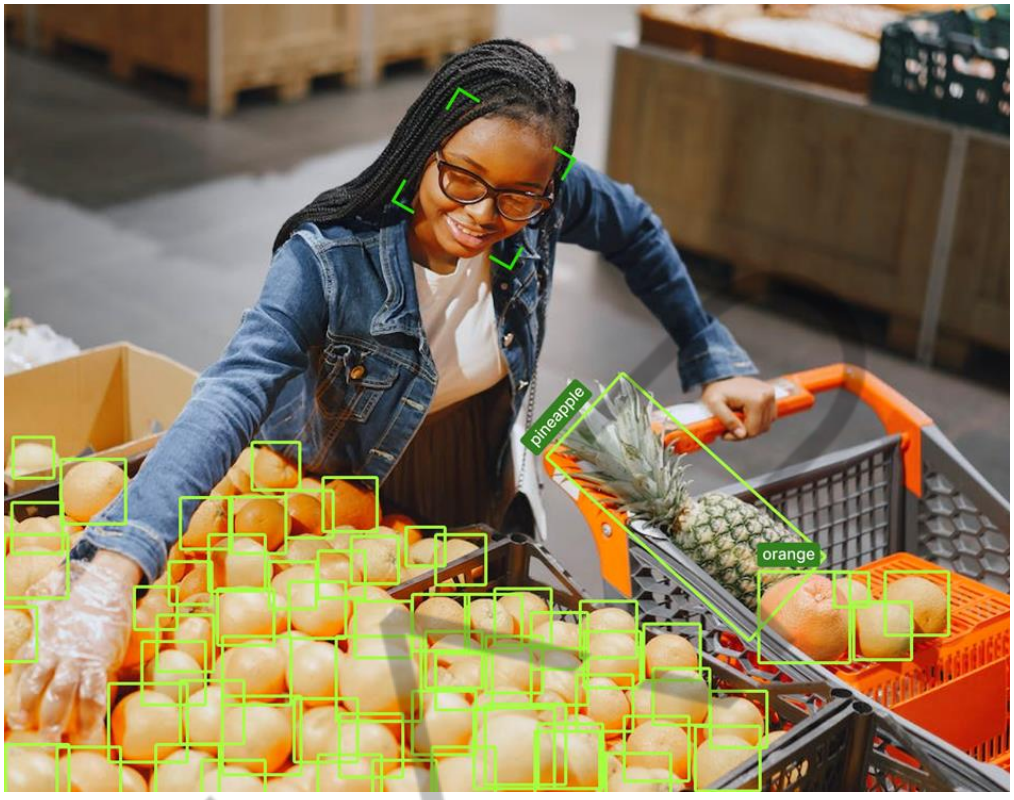


Figura 7- Exemplo de análise de comportamento de compra
Fonte: https://www.plugger.ai/blog/computer-vision-in-retail-top-9-applications

- **Saúde**: monitoramento de pacientes e assistência robótica baseada na detecção de gestos e movimentos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Visão computacional (Deteção de faces e objetos)

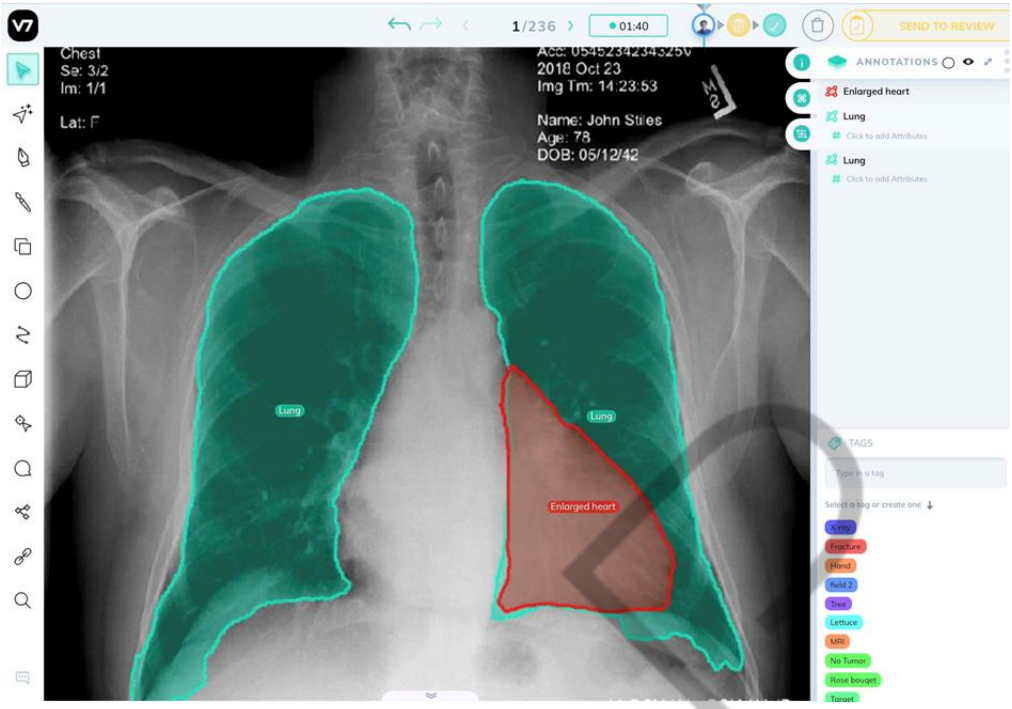


Figura 8 - Exemplo de diagnóstico por imagem
Fonte: https://www.v7labs.com/blog/what-is-computer-vision

Agora que você conhece a teoria por trás da visão computacional, vamos falar sobre seu lado prático.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 APLICAÇÃO DE MÉTODOS CLÁSSICOS DE DETECÇÃO DE FACES E OBJETOS

Nesta seção, vamos aplicar conceitos de processamento de imagens digitais para detecção de faces e objetos por meio de exemplos práticos. Vamos utilizar basicamente Python e a biblioteca OpenCV.

2.1 Detecção de Objetos por meio de detecção de contornos

Neste exemplo, utilizaremos a captura contínua de vídeo de uma webcam, processando cada quadro para identificar e ressaltar o objeto de interesse. A técnica empregada será a segmentação por cor no espaço HSV, que é menos sensível a variações de iluminação. Após a detecção das bordas, utilizaremos contornos para delinear claramente o objeto na imagem. Além de destacar visualmente o objeto, calcularemos e mostraremos seu centro de massa, fornecendo dados precisos sobre sua localização na cena. A Figura 9 ilustra o resultado da detecção, mostrando o objeto detectado com um contorno verde, destacando-o contra o fundo em vídeos em tempo real.

Figura 9 - Exemplo de segmentação de objeto
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
import cv2
import numpy as np

# Inicializa a captura de vídeo da webcam
cap = cv2.VideoCapture(0)

# Definição dos valores mínimo e máximo da máscara para a objeto
em HSV. Esse valor deve ser definido para cada objeto que se deseja
detectar.
objeto_lower1 = np.array([175, 120, 150])
objeto_upper1 = np.array([180, 255, 255])
objeto_lower2 = np.array([0, 140, 160])
objeto_upper2 = np.array([15, 255, 255])

# Loop para ler os frames da webcam
while True:
    ret, img = cap.read()
    if not ret:
        break

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    # Cria as máscaras para as faixas de vermelho
    mask_1 = cv2.inRange(img_hsv, objeto_lower1, objeto_upper1)
    mask_2 = cv2.inRange(img_hsv, objeto_lower2, objeto_upper2)

    # Combina as duas máscaras com uma operação bitwise_or
    mask_objeto = cv2.bitwise_or(mask_1, mask_2)

    # Encontra os contornos na imagem
    contours, _ = cv2.findContours(mask_objeto,
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    # Encontra o contorno de maior área
    max_area = 0
    max_index = -1
    for i, contour in enumerate(contours):
        area = cv2.contourArea(contour)
        if area > max_area:
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
                max_area = area
                max_index = i

        # Desenha o contorno de maior área na imagem original
        if max_index != -1:
            cv2.drawContours(img, [contours[max_index]], -1, (0, 255,
0), 10)

            # Calcula o centro de massa
            M = cv2.moments(contours[max_index])
            if M["m00"] != 0:
                cx = int(M["m10"] / M["m00"])
                cy = int(M["m01"] / M["m00"])
                # Desenha o centro de massa
                cv2.line(img, (cx - 20, cy), (cx + 20, cy), (128,
128, 0), 5)
                cv2.line(img, (cx, cy - 20), (cx, cy + 20), (128,
128, 0), 5)
                cv2.putText(img, f'Centro do objeto em Cx: {cx}, Cy:
{cy})', (0, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 0, 0), 3,
cv2.LINE_AA)

        # Exibe a imagem com o contorno de maior área
        cv2.imshow('Achei você!', img)

        # Sai do loop se a tecla 'q' for pressionada
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    # Libera a captura e fecha todas as janelas
    cap.release()
    cv2.destroyAllWindows()
```

Código-fonte 1 – Detecta objetos
Fonte: autor

2.2 Detecção de Faces com Haar Cascade

Neste exemplo, vamos continuar utilizando a captura contínua de vídeo de uma webcam, para identificar rapidamente faces. O detector de face de Viola-Jones é uma

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

referência obrigatória em qualquer curso ou livro de visão computacional. Desenvolvido em 2001, esse método foi pioneiro e disruptivo, permitindo pela primeira vez a detecção e o reconhecimento de rostos em tempo real, com alta eficiência. Isso foi alcançado por meio de uma técnica conhecida como "classificação em cascata", que rapidamente descarta áreas da imagem que não contêm faces, focando os recursos computacionais em regiões potenciais onde faces podem estar presentes.

O algoritmo utiliza características Haar, que são semelhantes aos padrões de onda de Haar, para identificar diferenças de intensidade de pixel que tipicamente correspondem a características faciais humanas. Por exemplo, a região dos olhos geralmente é mais escura que as regiões adjacentes da testa e das bochechas. O algoritmo aplica essas características em várias escalas e locais da imagem, usando uma série de classificadores em cascata, que rapidamente eliminam as regiões não promissoras enquanto concentram a análise nas regiões onde as características de face são mais prováveis de serem encontradas.

Esse método não só acelerou a detecção de faces em aplicações práticas, mas também estabeleceu um marco que guiou o desenvolvimento subsequente de técnicas mais avançadas no campo. Portanto, antes de mergulharmos nas complexidades do Deep Learning, é essencial apreciar e entender esse algoritmo fundamental.

Muito obrigado a Paul Viola e Michael Jones por suas contribuições significativas ao campo da visão computacional. :)

```
import cv2

# Carregar o classificador pré-treinado para detecção de faces
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')

# Inicializar a captura de vídeo da webcam
cap = cv2.VideoCapture(0)

# Loop para processar cada frame capturado pela webcam
while True:
   ret, frame = cap.read()
   if not ret:
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
        break

    # Converter a imagem capturada para escala de cinza
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detecção de faces na imagem
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1,
minNeighbors=5)

    # Desenhar retângulos ao redor das faces detectadas
    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 0), 2)

    # Exibir o resultado
    cv2.imshow('Face Detection', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Liberação da captura e fechamento das janelas
cap.release()
      cv2.destroyAllWindows()
```

Código-fonte 2 – Detecção de faces com Haar Cascade
Fonte: autor

Dica: para mais informações técnicas e um aprofundamento na leitura, consulte a publicação do artigo e o tutorial oficial da OpenCV.

- https://www.cs.cmu.edu/~efros/courses/LBMV07/Papers/viola-cvpr-01.pdf

- https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3 APLICAÇÃO DE APRENDIZADO PROFUNDO PARA DETECÇÃO DE FACES E OBJETOS

Nesta seção, vamos aplicar técnicas de Deep Learning com redes neurais convolucionais (CNNs) voltadas para problemas de visão computacional por sua capacidade de capturar características hierárquicas das imagens, utilizando a biblioteca TensorFlow e modelos pré-treinados para tarefas de classificação de imagens, assim como treinamento customizado para problemas de detecção de objetos e faces.

3.1 Uso de Redes Pré-treinadas para Classificação

As redes neurais convolucionais (CNNs) pré-treinadas permitem que desenvolvedores e pesquisadores, utilizem arquiteturas e modelos eficientes que já foram validados em extensos conjuntos de dados. O dataset **ImageNet** é uma base de dados utilizada para esses treinamentos. Composta por mais de 14 milhões de imagens categorizadas em mais de 20.000 categorias, a ImageNet foi projetada para uso em pesquisas de reconhecimento de objetos e tornou-se uma referência para competições de visão computacional como o desafio anual ImageNet Large Scale Visual Recognition Challenge (ILSVRC), que aconteceu até o ano de 2017.

Modelos pré-treinados em ImageNet, como o VGG16, Resnet, MobileNet e tantos outros, não apenas são eficazes para tarefas de classificação, como também uma base para tarefas mais complexas de visão computacional como detecção de faces e objetos.

No TensorFlow, esses modelos estão disponíveis para uso imediato, facilitando o acesso a tecnologias avançadas de visão computacional sem a necessidade de grande poder computacional ou longos períodos de treinamento.

Dica: para mais informações técnicas sobre modelos pré-treinados, consulte a documentação oficial.

- https://keras.io/api/applications/

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O exemplo do Código-fonte 3 realiza a implementação do modelo VGG16, treinado no dataset ImageNet, sendo carregado diretamente do Keras. A imagem de teste é obtida da internet e salva como bike.jpg, onde é carregada com o tamanho de entrada esperado pelo VGG16 (224x224 pixels). Esse redimensionamento é essencial para garantir que a arquitetura da rede, que foi treinada com imagens desse tamanho específico, possa aplicar corretamente seus filtros e pesos aprendidos durante o treinamento.

Após ser carregada, a imagem é convertida em um array numpy. Esse passo transforma a imagem de um formato que é compatível com bibliotecas de manipulação de imagens para um formato que pode ser manipulado matematicamente e processado por redes neurais.

A imagem então passa por um pré-processamento usando a função preprocess_input fornecida pelo Keras para o modelo VGG16. Esse pré-processamento ajusta os valores dos pixels da imagem de acordo com os critérios usados quando o VGG16 foi originalmente treinado. Isso geralmente inclui operações como normalização dos pixels, onde os valores de RGB são ajustados para faixas que o modelo espera, melhorando assim a eficácia da detecção e classificação.

De forma geral, as redes neurais frequentemente requerem uma dimensão adicional que representa o "lote" de imagens a ser processado. Isso é feito porque as redes neurais são otimizadas para processar múltiplas imagens em paralelo, e a dimensão do lote permite que a rede saiba quantas imagens estão sendo processadas de uma vez. Mesmo se você estiver processando apenas uma imagem, é necessário adicionar essa dimensão de lote usando np.expand_dims(x, axis=0). Isso altera o formato de entrada para (1, 224, 224, 3), onde '1' indica que há uma única imagem no lote.

```
## faz o download de uma imagem de teste da internet

!wget
https://images.tcdn.com.br/img/img_prod/777105/bicicleta_29_hope_21_ve
locidades_shimano_freios_disco_tamanho_17_12475_1_ac0b7c63eee851b87bcc
9832033c9826.jpg -O /content/bike.jpg
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
## import das bibliotecas
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input, decode_predictions

# Carrega o modelo VGG16 pré-treinado
model = VGG16()

# Caminho para a imagem local (substitua pela sua imagem local se necessário)
img_path = 'bike.jpg'

# Carrega e redimensiona a imagem para o tamanho esperado pelo VGG16 (224x224)
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)  # Converte para um array para exibir o shape

# Converte a imagem para um array NumPy e expande as dimensões para se adequar ao formato de entrada do modelo
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)
print(f"Transformação de imagem: De formato original: {img_array.shape} para: {x.shape}")

# Realiza a previsão usando o modelo VGG16
preds = model.predict(x)

# Decodifica as previsões em rótulos legíveis com a função decode_predictions
```

FIAP

```
# 'top=1' retorna a previsão mais provável
decoded_preds = decode_predictions(preds, top=1)[0]
predicted_label = decoded_preds[0][1]  # Obtém o rótulo predito
predicted_score = decoded_preds[0][2]  # Obtém a pontuação/confiança da
previsão

# Configuração do plot usando subplots para mostrar a imagem original
e o rótulo predito
fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(img)
ax[0].set_title("Imagem Original")
ax[0].axis('off')  # Desativa os eixos do primeiro subplot

# Exibição do rótulo predito no segundo subplot
ax[1].imshow(img)
ax[1].set_title(f"Predição:            {predicted_label}            -
{predicted_score*100:.2f}%")
ax[1].axis('off')  # Desativa os eixos do segundo subplot

plt.show()
```

Código-fonte 3 – Exemplo modelo VGG16 pré-treinada
Fonte: autor

O método `predict` do modelo é chamado com a imagem de entrada x, que agora está formatada corretamente. O modelo utiliza sua arquitetura treinada para analisar a imagem e fazer previsões sobre o que a imagem representa.

O resultado, `preds`, é um array de probabilidades que indica a confiança do modelo sobre as possíveis categorias para a imagem de entrada.

A função `decode_predictions` transforma o array de probabilidades preds em uma lista de categorias e probabilidades mais `legíveis`. O parâmetro `top=1` especifica que estamos interessados apenas na categoria mais provável, podemos

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

alterar esse parâmetro para capturar mais predições como `top=3` ou `top=5` para as 3 ou 5 categorias mais prováveis.

Cada entrada na lista decoded_preds contém:

- ID do ImageNet;
- Nome da categoria;
- Confiança (score) de que a imagem pertence a essa categoria.

Ao final, é realizada a predicted_label, que armazena o nome da categoria que o modelo identificou como mais provável para a imagem, enquanto predicted_score reflete a confiança do modelo nessa previsão. A Figura 10 exibe, à esquerda, a imagem de teste e à direta, a saída predita com a classificação de mountain_bike com 81,79% de confiança.

Figura 10 - Exemplo de classificação VGG16
Fonte: autor

3.2 Alterando o Modelo da rede Pré-treinada para Classificação

Alterar o código para usar um modelo pré-treinado diferente, como o MobileNet, Resnet, inception, em vez do VGG16, é um processo bastante simples com o TensorFlow. Vamos ver como isso pode ser feito com algumas pequenas alterações no código existente.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Alterando o Modelo para MobileNet.
    1. Importar o Modelo MobileNet: primeiro, você precisa importar o MobileNet do TensorFlow Keras, assim como fizemos com o VGG16.

```
from tensorflow.keras.applications import MobileNet
```

Código-fonte 4 – Carrega modelo MobileNet
Fonte: autor

2. Carregar o Modelo MobileNet: substitua a instância do modelo VGG16 por MobileNet. Como o MobileNet também foi treinado usando o dataset ImageNet, você pode utilizar a mesma função de pré-processamento e decodificação.

```
model = MobileNet(weights='imagenet')
```

Código-fonte 5 – Carrega MobileNet
Fonte: autor

3. Restante do Código: o restante do código, incluindo o carregamento da imagem, pré-processamento e decodificação das previsões, permanece o mesmo. Isso ocorre porque os métodos preprocess_input e decode_predictions são projetados para serem compatíveis com todos os modelos pré-treinados disponíveis no Keras que usam o ImageNet.

A Figura 11 exibe o resultado de classificação obtido com o modelo MobileNet. Note que a saída predita é mountain_bike mas com 99,22% de confiança, bem superior à confiança obtida do modelo VGG16.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 11 - Exemplo de classificação MobileNet
Fonte: autor

3.3 Adaptando Redes de Classificação para Detecção de Objetos

Já sabemos como classificar objetos, mas como podemos detectar faces e objetos? Vamos explorar como transformar redes de classificação, originalmente projetadas para identificar o que aparece em uma imagem, em detectores capazes de localizar onde os objetos estão na imagem. Isso envolve não apenas reconhecer os objetos, mas também determinar suas posições e dimensões dentro da imagem.

3.4 Do Classificador ao Detector

Transformar uma rede de classificação em uma de detecção geralmente requer modificações significativas na arquitetura da rede e no processo de treinamento. A seguir, vamos detalhar esse passo a passo na prática, dividido em 9 etapas.

Dica: Notebook com o código completo pode ser encontrado em:

https://colab.research.google.com/drive/1cBJj3Wq-0U47HXXtuLx4EYijEMoCLhzV?usp=sharing

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Etapa 1: Seleção e Download do Conjunto de Dados.

Para começar, vamos baixar um subconjunto de dados do dataset CALTECH-101, usado em pesquisas de visão computacional. Esse subconjunto específico inclui imagens pertencentes a três categorias apenas: elefantes, borboletas e rostos de puma.

Contamos com 64 imagens de elefantes, 88 de borboletas e 66 de rostos de puma. Cada imagem desse subconjunto é acompanhada por rótulos de classe e coordenadas de caixas delimitadoras (bounding box) em formato .MAT (Matlab), essas anotações são essenciais para tarefas de detecção de objetos.

```
from zipfile import ZipFile


# Download do dataset
!wget -O CALTECH.zip https://zenodo.org/record/4126613/files/CALTECH.zip?download=1


# Extrai conteúdo do zip
with ZipFile('CALTECH.zip', 'r') as zipObj:
  zipObj.extractall()
```

Código-fonte 6 – Carrega dados
Fonte: autor

Organização dos Dados: certifique-se de que as imagens estejam bem-organizadas em pastas correspondentes às suas classes (elefante, borboleta, rosto de puma). Verifique os arquivos .MAT para as coordenadas das caixas delimitadoras, conforme mostra a imagem a seguir, onde a pasta CALTECH_Annotations são os arquivos .MAT e CALTECH_Dataset é o diretório de imagens :

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

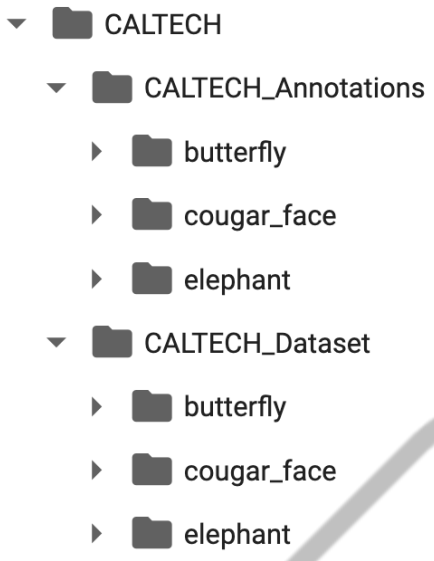


Figura 12– Organização dos dados
Fonte: autor

Etapa 2: Enriquecimento de Dados via Aumento (Data Augmentation).

Não possuímos muitas imagens de cada classe de objeto, isso pode prejudicar o treinamento, é nesse ponto que podemos utilizar o aumento de dados, que é uma técnica para expandir um conjunto de dados e fazer com que um modelo generalize melhor.

Embora o keras/tensorflow tenha uma função ImageDataGenerator para realizar essa tarefa, não podemos utilizar nesse caso, pois ao realizar o aumento de dados, empregamos técnicas como deslocamento, rotação, cisalhamento, corte, zoom etc. Todas essas técnicas alteram fundamentalmente a imagem, mas quando a imagem muda, as coordenadas correspondentes das caixas delimitadoras também devem mudar. Como a função de aumento do Keras modifica apenas a imagem e não as coordenadas das caixas, é uma ideia muito ruim usar a função de aumento padrão.

Vamos utilizar a biblioteca como **imgaug**, que permite aumentar imagens e localizações de caixas delimitadoras.

Essa etapa envolve diversos passos, vamos entender cada um deles:

- Estruturar dados.
- Definir tipos data augmentation.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Aplicar data augmentation nas imagens e rótulos.

Estruturar dados

Vamos começar a estruturar os dados de forma tabular, facilitando manipulações subsequentes, vamos converter as anotações do formato .MAT para o formato .csv. O código a seguir vai:

- Percorrer os diretórios onde estão armazenadas as imagens e as anotações correspondentes;
- Extrair Informações de cada imagem e sua anotação:
  - Altura da imagem;
  - Largura da imagem;
  - Coordenadas da caixa delimitadora (bounding box);
  - Classe do objeto na imagem.
- Armazenar os dados em um dataframe do pandas.

```
import os
import cv2
import scipy
import pandas as pd

# Função para Obter atributos para cada imagem
def extract_mat_contents(annot_directory, image_dir):

        # Criar um Parser MAT
        mat = scipy.io.loadmat(annot_directory)

        # Obter a altura e largura da nossa imagem
        height, width = cv2.imread(image_dir).shape[:2]

        # Obter as coordenadas da caixa delimitadora
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
        x1, y2, y1, x2 = tuple(map(tuple, mat['box_coord']))[0]

        # Dividimos o diretório da imagem passado no método e escolhemos
o índice
        # do nome da pasta que corresponde à sua classe
        class_name = image_dir.split('/')[2]

        # Formatar o nome do arquivo
        filename = '/'.join(image_dir.split('/')[-2:])

        # Retornar os atributos extraídos
        return filename, width, height, class_name, x1, y1, x2, y2


# Função para converter arquivos MAT em CSV
def mat_to_csv(annot_directory, image_directory, classes_folders):

 # Lista contendo todos os nossos atributos referentes a cada imagem
 mat_list = []

 # Percorremos cada classe e suas etiquetas uma a uma para pré-processar
e aumentar
 for class_folder in classes_folders:

   # Definir nossos diretórios de imagens e anotações
   image_dir = os.path.join(image_directory, class_folder)
   annot_dir = os.path.join(annot_directory, class_folder)

   # Obter cada arquivo nos diretórios de imagem e anotação
   mat_files = sorted(os.listdir(annot_dir))
   img_files = sorted(os.listdir(image_dir))
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    # Percorrer cada imagem e sua etiqueta
    for mat, image_file in zip(mat_files, img_files):

        # Caminho completo do arquivo MAT
        mat_path = os.path.join(annot_dir, mat)

        # Caminho completo da imagem
        img_path = os.path.join(image_dir, image_file)

        # Obter atributos para cada imagem
        value = extract_mat_contents(mat_path, img_path)

        # Adicionar os atributos à lista mat_list
        mat_list.append(value)

    # Colunas para o DataFrame do Pandas
    column_name = ['filename', 'width', 'height', 'class', 'xmin', 'ymin',
                   'xmax', 'ymax']

    # Criar o DataFrame a partir de mat_list
    mat_df = pd.DataFrame(mat_list, columns=column_name)

    # Retornar o dataframe
    return mat_df

# As Classes que usaremos para nosso treinamento
classes_list = sorted(['butterfly', 'cougar_face', 'elephant'])

# Definir nossos diretórios de imagens e anotações
image_directory = 'CALTECH/CALTECH_Dataset'
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
annot_directory = 'CALTECH/CALTECH_Annotations'


# Executar a função para converter todos os arquivos MAT em um DataFrame
do Pandas
labels_df = mat_to_csv(annot_directory, image_directory, classes_list)


# Salvar o DataFrame do Pandas como arquivo CSV
labels_df.to_csv(('labels.csv'), index=None)
```

Código-fonte 7 – Estrutura os dados csv
Fonte: autor

É esperado que o arquivo `label.csv` seja semelhante à imagem a seguir, com os mesmos atributos.

labels.csv

1 to 10 of 218 entries Filter

filename	width	height	class	xmin	ymin	xmax	ymax
butterfly/image_0001.jpg	300	198	butterfly	11	3	284	188
butterfly/image_0003.jpg	300	192	butterfly	4	39	272	192
butterfly/image_0004.jpg	300	271	butterfly	10	9	289	264
butterfly/image_0005.jpg	300	278	butterfly	45	55	299	204
butterfly/image_0006.jpg	300	271	butterfly	50	20	299	265
butterfly/image_0007.jpg	300	200	butterfly	40	38	276	183
butterfly/image_0008.jpg	300	252	butterfly	29	23	298	237
butterfly/image_0009.jpg	300	253	butterfly	24	29	273	231
butterfly/image_0010.jpg	300	200	butterfly	22	28	266	176
butterfly/image_0011.jpg	300	227	butterfly	26	43	258	209

Show 10 per page 1 2 10 20 22

Figura 13 - Organização dos dados csv
Fonte: autor

Agora que temos os dados prontos no formato necessário, podemos prosseguir com o aumento de dados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Definir tipos data augmentation

Vamos iniciar a biblioteca imgaug com as especificações que iremos aplicar nas imagens. O imgaug seleciona n tipos aleatórios da lista abaixo e aplica a uma imagem. Para cada imagem, uma nova combinação aleatória da lista é utilizada. O valor de n está definido para 2, mas pode ser alterado.

Vamos aplicar 7 tipos diferentes de aumentos ao nosso conjunto de dados:

- Escala (Scale): a imagem é redimensionada para aumentar ou diminuir, variando de 50% a 150% do seu tamanho original. Isso simula diferentes distâncias dos objetos em relação à câmera;
- Rotação (Rotate): a imagem é rotacionada entre -60 e 60 graus. Isso ajuda o modelo a reconhecer objetos em várias orientações;
- Translação (Shift): a imagem é deslocada horizontalmente ou verticalmente em até 30% do tamanho da imagem. Isso é útil para treinar o modelo a identificar objetos que não estão centralizados;
- Espelhamento Horizontal (Flip): a imagem é espelhada horizontalmente. Esse tipo de augmentação é comum para aumentar a diversidade de dados sem alterar a natureza dos objetos;
- Brilho (Brightness): o brilho da imagem é ajustado, aumentando ou diminuindo dentro de um intervalo de 50% a 150%. Isso prepara o modelo para operar sob diferentes condições de iluminação;
- Desfoque Gaussiano (Gaussian Blur): um desfoque gaussiano é aplicado à imagem, com um sigma variando de 1,0 a 3,0. Isso simula o efeito de diferentes qualidades de câmera e condições de foco;
- Ruído Gaussiano (Gaussian Noise): adiciona ruído gaussiano à imagem, simulando o efeito de grão em imagens resultantes de alta sensibilidade ISO em câmeras.

```
import imgaug.augmenters as iaa


# Definição das augmentações que você deseja aplicar ao seu conjunto de
dados
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Estamos definindo que 'n' augmentações aleatórias sejam 2.
n = 2
image_augmentations = iaa.SomeOf(n, [
   iaa.Affine(scale=(0.5, 1.5)),  # Escala
   iaa.Affine(rotate=(-60, 60)),  # Rotação
   iaa.Affine(translate_percent={"x": (-0.3, 0.3), "y": (-0.3, 0.3)}),
# Translação
   iaa.Fliplr(1),  # Espelhamento Horizontal
   iaa.Multiply((0.5, 1.5)),  # Ajuste de Brilho
   iaa.GaussianBlur(sigma=(1.0, 3.0)),  # Desfoque Gaussiano
   iaa.AdditiveGaussianNoise(scale=(0.03*255,  0.05*255))     #  Ruído
Gaussiano
])
```

Código-fonte 8 – Função auxiliar bounding box
Fonte: autor

Aplicar data augmentation nas imagens e rótulos

Vamos aproveitar para criar uma função auxiliar que converterá os objetos de caixa delimitadora da biblioteca imgaug para um DataFrame do pandas. Isso facilitará a manipulação e análise dos dados de localização dos objetos nas imagens.

```
# Função para converter o bounding box da imagem em DataFrame
def bounding_boxes_to_df(bounding_boxes_object):

   # Converte Bounding Boxes para Array
   bounding_boxes_array = bounding_boxes_object.to_xyxy_array()

   # Converte o array em DataFrame
   df_bounding_boxes = pd.DataFrame(bounding_boxes_array,
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
                                    columns=['xmin',     'ymin',     'xmax',
'ymax'])

    # Retorna o DataFrame
    return df_bounding_boxes
```

Código-fonte 9 – Função converte bounding box
Fonte: autor

E finalmente realizar as aumentações de imagem tanto nas imagens quanto nas caixas delimitadoras. Ela lerá o dataframe de etiquetas que criamos anteriormente para obter as informações da caixa delimitadora de cada imagem e conforme for aumentando, também editará as coordenadas da caixa delimitadora, de modo que as coordenadas permaneçam corretas mesmo após a aumentação ser concluída.

```
# Função que aplica data augmentation
def image_aug(df, images_path, aug_images_path, augmentor, multiple=3):

    # Preencher este DataFrame com atributos de imagem
    augmentations_df = pd.DataFrame(
        columns=['filename', 'width', 'height', 'class', 'xmin', 'ymin',
'xmax', 'ymax'])

    # Agrupar os dados por nomes de arquivo
    grouped_df = df.groupby('filename')

    # Criar o diretório para todas as imagens aumentadas
    if not os.path.exists(aug_images_path):
        os.mkdir(aug_images_path)

    # Criar diretórios para cada classe de imagens aumentadas
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    for folder in df['class'].unique():
        if not os.path.exists(os.path.join(aug_images_path, folder)):
            os.mkdir(os.path.join(aug_images_path, folder))

    for i in range(multiple):

        # Sufixo que adicionamos a cada diferente aumento de uma imagem
        image_postfix = str(i)

        # Laço para realizar as aumentações
        for filename in df['filename'].unique():

            augmented_path = os.path.join(aug_images_path, filename) +
image_postfix + '.jpg'

            # Pegar uma imagem por vez com suas informações
            single_image = grouped_df.get_group(filename)
            single_image = single_image.reset_index()
            single_image = single_image.drop(['index'], axis=1)

            # Ler a imagem
            image = imageio.imread(os.path.join(images_path, filename))

            # Obter a caixa delimitadora
            bounding_box_array = single_image.drop(['filename', 'width',
'height', 'class'], axis=1).values

            # Fornecer a caixa delimitadora para a biblioteca imgaug
            bounding_box                                              =
BoundingBoxesOnImage.from_xyxy_array(bounding_box_array,
shape=image.shape)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
            # Realizar 2 aumentações aleatórias
            image_aug,    bounding_box_aug    =    augmentor(image=image,
bounding_boxes=bounding_box)

            # Descartar a caixa delimitadora que sai completamente da
imagem
            bounding_box_aug = bounding_box_aug.remove_out_of_image()

            # Cortar a caixa delimitadora que está parcialmente fora da
imagem
            bounding_box_aug = bounding_box_aug.clip_out_of_image()

            # Descartar a imagem se a caixa delimitadora foi descartada
            if    re.findall('Image...',    str(bounding_box_aug))    ==
['Image([]']:
                pass

            else:

                # Criar o arquivo de imagem aumentada
                imageio.imwrite(augmented_path, image_aug)

                #  Atualizar  a  largura  e  altura  da  imagem  após  a
aumentação
                info_df  =  single_image.drop(['xmin',  'ymin',  'xmax',
'ymax'], axis=1)
                for index, _ in info_df.iterrows():
                    info_df.at[index, 'width'] = image_aug.shape[1]
                    info_df.at[index, 'height'] = image_aug.shape[0]

                # Adicionar o prefixo a cada imagem para diferenciar se
necessário
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
                info_df['filename']  =  info_df['filename'].apply(lambda
x: x + image_postfix + '.jpg')


                # Criar o DataFrame de caixas delimitadoras aumentadas

                bounding_box_df                                        =
bounding_boxes_to_df(bounding_box_aug)


                # Concatenar os nomes dos arquivos, altura, largura e
caixas delimitadoras

                aug_df = pd.concat([info_df, bounding_box_df], axis=1)


                # Adicionar todas as informações ao augmentations_df que
inicializamos acima

                augmentations_df = pd.concat([augmentations_df, aug_df])


    # Remover índice

    augmentations_df = augmentations_df.reset_index()

    augmentations_df = augmentations_df.drop(['index'], axis=1)


    # Retornar o DataFrame

    return augmentations_df


augmented_images_df     =     image_aug(labels_df,     image_directory,
'aug_images', image_augmentations)
```

Código-fonte 10 – Função data augmentation
Fonte: autor

Vamos avaliar o resultado da etapa de data augmentation. Repare que agora temos o diretório aug_images com as imagens criadas e aug.csv com a descrição dos labels.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
augmented_images_df = augmented_images_df.sort_values('filename',
ignore_index=True)

augmented_images_df.to_csv('aug.csv')


# Verificar o Tamanho do Conjunto de Dados

print('O tamanho total do nosso conjunto de dados antes das aumentações
era: ', len(labels_df))

print('O tamanho total do nosso conjunto de dados após as aumentações
é: ', len(augmented_images_df))



# O tamanho total do nosso conjunto de dados antes das aumentações era:
218

# O tamanho total do nosso conjunto de dados após as aumentações é:  654
```

Código-fonte 11 – Verifica dataset apõs data augmentation
Fonte: autor

O dataset foi triplicado onde cada imagem recebeu, de forma aleatória, 3 transformações de aumento de dados. Como mostra a figura a seguir, com a imagem image_0001.jpg que foi aumentada para image_0001.jpg0.jpg, image_0001.jpg1.jpg e image_0001.jpg2.jpg.

	filename	width	height	class	xmin	ymin	xmax	ymax
0	butterfly/image_0001.jpg0.jpg	300	198	butterfly	0.0	0.0	200.58783	149.40422
1	butterfly/image_0001.jpg1.jpg	300	198	butterfly	0.0	22.236809	225.43173	198.0
2	butterfly/image_0001.jpg2.jpg	300	198	butterfly	16.0	3.0	289.0	188.0
3	butterfly/image_0003.jpg0.jpg	300	192	butterfly	0.0	8.982044	260.0547	161.98204
4	butterfly/image_0003.jpg1.jpg	300	192	butterfly	61.679485	55.926903	300.0	192.0
5	butterfly/image_0003.jpg2.jpg	300	192	butterfly	28.0	39.0	296.0	192.0
6	butterfly/image_0004.jpg0.jpg	300	271	butterfly	0.0	0.0	300.0	271.0
7	butterfly/image_0004.jpg1.jpg	300	271	butterfly	11.0	9.0	290.0	264.0
8	butterfly/image_0004.jpg2.jpg	300	271	butterfly	0.0	8.905224	263.1812	269.0585
9	butterfly/image_0005.jpg0.jpg	300	278	butterfly	45.0	55.0	299.0	204.0

Figura 14 - Organização dos dados csv
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Etapa 3: Pré-processamento do Conjunto de Dados.

Com o conjunto de dados criado, podemos realizar a etapa de pré-processamento, basicamente preparar as **imagens** e as **anotações** para o treinamento do modelo de detecção de objetos.

Pré-processamento da Imagem

As imagens serão pré-processadas por meio de vários passos:

- Redimensionamento: todas as imagens são redimensionadas para um tamanho fixo. Esse tamanho uniforme é necessário para que as imagens possam ser processadas de maneira eficiente pela rede neural, que espera entradas de dimensões fixas;
- Conversão para Formato de Canal RGB: garante que todas as imagens estejam no formato de canal RGB, que é o formato padrão para o processamento de imagens coloridas;
- Normalização: as imagens são normalizadas dividindo cada pixel por 255.0. Essa etapa transforma os valores dos pixels, originalmente no intervalo de 0 a 255, para um intervalo de 0 a 1. A normalização ajuda a estabilizar a aprendizagem e melhora a convergência durante o treinamento, pois os valores de entrada menores e mais uniformes são geralmente mais fáceis de manipular pelos pesos da rede.

Pré-processamento das anotações

- Conversão de rótulos: os rótulos de classe são convertidos em vetores codificados em one-hot. É importante para a classificação, pois transforma os rótulos categóricos em um formato que pode ser facilmente processado pelos algoritmos de aprendizado de máquina;
- Escalonamento das Caixas Delimitadoras: as coordenadas das caixas delimitadoras serão escalonadas para o intervalo de 0 a 1. Essa normalização é importante porque facilita para a rede prever valores em uma faixa fixa para cada imagem.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
def preprocess_dataset(pasta_imagens, lista_classes, df,
tamanho_imagem=300):

    # Listas que conterão todo o conjunto de dados
    rotulos = []
    caixas = []
    lista_imagens = []

    # Obtém a altura e largura de cada imagem no DataFrame
    h = df['height']
    w = df['width']

    # Cria uma cópia dos rótulos no DataFrame
    rotulos = list(df['class'])
    # Converte rotulos para inteiros, e one hot encode
    label_encoder = LabelEncoder()
    integer_labels = label_encoder.fit_transform(labels)
    onehot_labels = to_categorical(integer_labels)

    # Cria uma cópia dos valores das caixas delimitadoras e os normaliza
    for x1, y1, x2, y2 in zip(list(df['xmin']/w), list(df['ymin']/h),
                              list(df['xmax']/w), list(df['ymax']/h)):

        arr = [x1, y1, x2, y2]
        caixas.append(arr)

    # Itera sobre cada classe e seus rótulos
    for pasta_classe in lista_classes:

        # Define o diretório das imagens
        dir_imagens = os.path.join(pasta_imagens, pasta_classe)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
        # Arquivos de anotação e imagem
        arquivos_img = sorted(os.listdir(dir_imagens))

        # Itera sobre cada imagem e seu rótulo
        for arquivo_imagem in arquivos_img:

            # Caminho completo da imagem
            caminho_img = os.path.join(dir_imagens, arquivo_imagem)

            # Lê a imagem
            img = cv2.imread(caminho_img)

            # Redimensiona todas as imagens para um tamanho fixo
            imagem = cv2.resize(img, (tamanho_imagem, tamanho_imagem))

            # Converte a imagem de BGR para RGB, pois o modelo foi
treinado em imagens RGB
            imagem = cv2.cvtColor(imagem, cv2.COLOR_BGR2RGB)

            # Normaliza a imagem dividindo-a por 255.0
            imagem = imagem.astype("float") / 255.0

            # Adiciona à lista de imagens
            lista_imagens.append(imagem)

    return onehot_labels, caixas, lista_imagens

# Imagens com tamanho 300, 300
image_size = 300
```

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Chama função de pre-processamento
labels, boxes, img_list = preprocess_dataset('aug_images',
classes_list,
                                                augmented_images_df)
```

Código-fonte 12 – Continua código
Fonte: autor

Com os dados pré-processados, podemos embaralhar todos os dados do conjunto.

```
# Compacta e embaralhar os dados
combined_list = list(zip(img_list, boxes, onehot_labels))
random.shuffle(combined_list)

# Extrai novamente os dados embaralhados
img_list, boxes, labels = zip(*combined_list)
```

Código-fonte 13 – Embaralha dados
Fonte: autor

Etapa 4: Visualização de Dados e Anotações.

Vamos visualizar algumas imagens aleatórias do conjunto de dados com suas caixas delimitadoras, a fim de verificar a precisão das anotações e garantir que as etapas de pré-processamento estão corretas.

```
import matplotlib.pyplot as plt
import numpy as np
import cv2
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
import random


def    visualize_sample_with_bboxes(img_list,    boxes,    image_size,
num_samples=20):
   plt.figure(figsize=(20, 20))
   random_range = random.sample(range(1,len(img_list)), num_samples)

   for iteration, i in enumerate(random_range, 1):
       # Bounding box of each image
       a1, b1, a2, b2 = boxes[i]

       # Rescaling the bounding box values to match the image size
       x1, x2 = a1 * image_size, a2 * image_size
       y1, y2 = b1 * image_size, b2 * image_size

       # The image to visualize
       image = np.copy(img_list[i])   # Copia para evitar alteração na
lista original

       # Draw bounding boxes on the image
       cv2.rectangle(image, (int(x1), int(y1)), (int(x2), int(y2)), (0,
255, 0), 3)

       # Clip the values to 0-1 and draw the sample of images
       image = np.clip(image, 0, 1)
       # Draw the sample of images
       plt.subplot(4, 5, iteration)
       plt.imshow(image)
       plt.axis('off')

   plt.show()
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Chamando a função
visualize_sample_with_bboxes(img_list,      boxes,      image_size=300,
num_samples=20)
```

Código-fonte 14 – exibe resultado
Fonte: autor

É esperado como resultado a exibição de 20 imagens aleatórias do dataset com suas respectivas caixas delimitadoras. Como a imagem a seguir.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

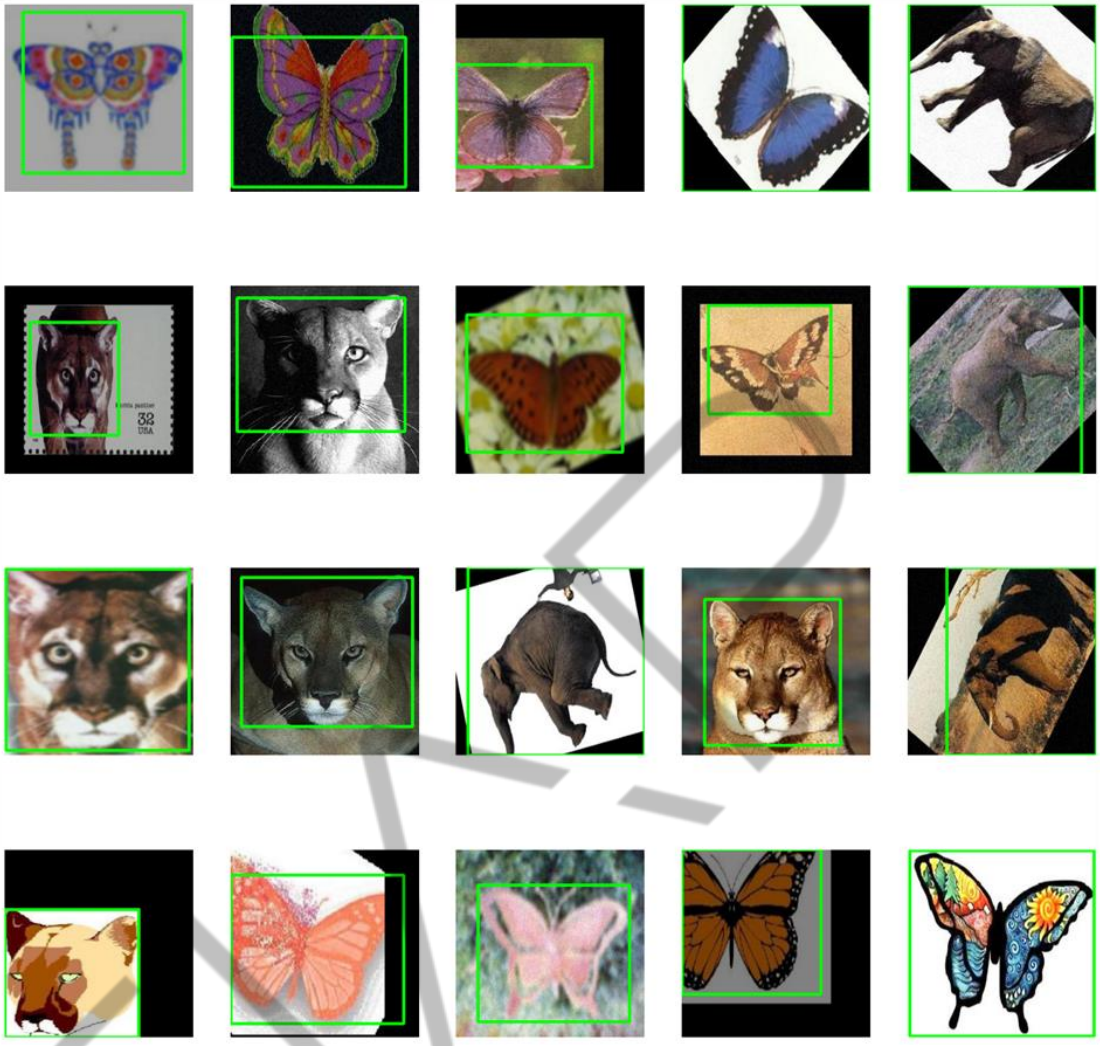


Figura 15 - exemplo dataset
Fonte: autor

Etapa 5: Divisão do Conjunto de Dados em Treino e Validação.

Com a criação e preparação dos dados, podemos dividir o conjunto de dados de maneira que o modelo possa ser treinado e validado eficientemente.

```
from sklearn.model_selection import train_test_split


# Divisão dos dados de imagens, rótulos e suas anotações
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
train_images, val_images, train_labels, \
val_labels, train_boxes, val_boxes = train_test_split(
   np.array(img_list), np.array(labels), np.array(boxes),
   test_size=0.2, random_state=42
)

print('Total de Imagens de Treinamento: {}, Total de Imagens de Validação: {}'.format(
   len(train_images), len(val_images)))
```

Código-fonte 15 – Separa dados testes treino
Fonte: autor

Etapa 6: Construção do Modelo com Múltiplas Saídas.

Para a construção da arquitetura do modelo de múltiplas saídas, será aplicado o aprendizado por transferência, processo em que adaptamos arquiteturas existentes adicionando camadas que podem prever não apenas classes, mas também localizações de objetos. Essa abordagem permite que aproveitemos os recursos avançados de um modelo pré-treinado, economizando tempo e recursos computacionais necessários para treinar um modelo complexo desde o início.

Vamos executar alguns passos:

- Escolhemos uma rede pré-treinada: selecionamos uma rede neural pré-treinada disponível no Keras (por exemplo, VGG16, ResNet50, InceptionV3…) com base nas características e requisitos do seu problema. Cada arquitetura tem suas próprias vantagens e desvantagens, portanto, escolha aquela que melhor se adapta às suas necessidades.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Dica: Para conhecer os modelos pré-treinados acesse:

https://keras.io/api/applications/

- Removemos a camada de classificação: carregamos a rede neural pré-treinada sem a camada de classificação final. Isso pode ser feito usando o argumento include_top=False ao carregar o modelo no Keras. Isso permitirá que você adicione suas próprias camadas personalizadas para classificar as novas categorias.

O modelo que vamos usar é o NasnetMobile. Esse é um modelo eficiente, com um bom equilíbrio entre velocidade e precisão.

```
# Carrega o modelo pre-treinado e exclui o top
N_mobile = tf.keras.applications.NASNetMobile( input_tensor = Input(
    shape=(image_size, image_size, 3)),
    include_top=False,
    weights='imagenet')
```

Código-fonte 16 – Carrega modelo
Fonte: autor

- Adição de camadas personalizadas: adicionamos camadas específicas para o problema de classificação e detecção de objetos;
- Congele as camadas pré-treinadas: é uma boa prática congelar os pesos das camadas pré-treinadas da rede neural, especialmente durante as primeiras épocas do treinamento. Isso evitará que os pesos dessas camadas sejam atualizados e preservem o conhecimento prévio que elas possuem. No Keras, fazemos isso com modelxxx.trainable = False.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O modelo personalizado terá uma saída de classificação e uma saída de detecção de objetos em paralelo. Usamos 'Global Average Pooling' para reduzir as dimensões das características de entrada, o que ajuda a controlar o overfitting e reduzir o número de parâmetros necessários no modelo. Adiciona várias camadas 'Dense' e 'Dropout' para construir a cabeça de classificação e a cabeça de localização. As camadas 'Dense' aprendem a partir das características aplainadas, enquanto as camadas 'Dropout' ajudam a evitar o overfitting ao desativar aleatoriamente parte das conexões durante o treinamento. Criamos dois 'outputs' no modelo: um para a classificação das imagens e outro para a localização dos objetos (caixas delimitadoras).

```
def create_model(numero_de_classes):

    # Congela o modelo inteiro
    N_mobile.trainable = False

    # Inicia pegando os mapas de características de saída do NASNETMobile
    saida_base_modelo = N_mobile.output

    # Converte para um vetor unidimensional usando 'Global Average
Pooling'.
    flattened_output = GlobalAveragePooling2D()(saida_base_modelo)

    # Cria a Cabeça de Classificação, a camada final contém
    # Unidades de saída = número de classes
    predicao_classe = Dense(256, activation="relu")(flattened_output)
    predicao_classe = Dense(128, activation="relu")(predicao_classe)
    predicao_classe = Dropout(0.2)(predicao_classe)
    predicao_classe = Dense(64, activation="relu")(predicao_classe)
    predicao_classe = Dropout(0.2)(predicao_classe)
    predicao_classe = Dense(32, activation="relu")(predicao_classe)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    predicao_classe = Dense(numero_de_classes, activation='softmax',
name="class_output")(predicao_classe)

    # Cria a Cabeça de Localização, a camada final contém 4 nodos para
x1, y1, x2, y2,
    # respectivamente.
    saida_caixa = Dense(256, activation="relu")(flattened_output)
    saida_caixa = Dense(128, activation="relu")(saida_caixa)
    saida_caixa = Dropout(0.2)(saida_caixa)
    saida_caixa = Dense(64, activation="relu")(saida_caixa)
    saida_caixa = Dropout(0.2)(saida_caixa)
    saida_caixa = Dense(32, activation="relu")(saida_caixa)
    predicoes_caixa = Dense(4, activation='sigmoid',
name="box_output")(saida_caixa)

    # Agora combinamos as duas cabeças
    modelo = Model(inputs=N_mobile.input, outputs=[predicoes_caixa,
predicao_classe])

    return modelo

# Cria o modelo para 3 classes: Elefante, Borboleta, Rosto de Puma
modelo = create_model(3)
```

Código-fonte 17 – Continua testes
Fonte: autor

Etapa 7: Compilação e Treinamento do Modelo.

Vamos compilar e treinar o modelo adaptado. Para isso, vamos utilizar dicionários em python para determinar os parâmetros de treinamento, lembrando que estamos definindo os parâmetros tanto para classificação como detecção. Utilizamos

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

diferentes tipos de perda que são usados para diferentes partes do modelo, refletindo as tarefas distintas de classificação e regressão, assim como as métricas definidas para avaliar o desempenho de cada tarefa. Usamos acurácia para a classificação e erro quadrático médio para avaliar a precisão das caixas delimitadoras.

Utilizamos as técnicas de Early Stopping e Reduce Learning Rate para otimizar o treinamento, evitando o overfitting e ajustando o learning rate para melhorar a aprendizagem se o modelo parar de melhorar.

```
# Define a loss
losses = {
   "box_output": "mean_squared_error",
   "class_output": "categorical_crossentropy"
   }

# Atribuição de pesos às perdas.
loss_weights = {
   "box_output": 1.0,
   "class_output": 1.0
   }

# Configura as métricas
metrics = {
   'class_output': 'accuracy',
   'box_output':  'mse'
   }

# Define Early Stopping
stop = EarlyStopping(monitor = "val_loss", min_delta = 0.0001, patience
= 40,
                    restore_best_weights = True
                     )
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Change the learning rate according to number of epochs to boost learning
reduce_lr = ReduceLROnPlateau(monitor = "val_loss", factor = 0.0002,
                              patience = 30, min_lr = 1e-7, verbose = 1)


# Compila o modelo com o otimizador SGD
model.compile(optimizer = SGD(lr = 1e-3, momentum = 0.9), loss = losses, loss_weights = loss_weights,
   metrics = metrics)
```

Código-fonte 18 – compila o modelo
Fonte: Autor

O método fit do tensorflow é utilizado para treinar o modelo. Passamos os valores definidos anteriormente e os dados de entrada.

```
# Treina o modelo
history = model.fit(x = train_images,
                    y= {
                        "box_output": train_boxes,
                        "class_output": train_labels
                        },
                    validation_data=(
                        val_images,
                        {
                          "box_output": val_boxes,
                          "class_output": val_labels
                          }), batch_size = 32, epochs = 500,
                    callbacks=[reduce_lr, stop])
```

Código-fonte 19 – Compila modelo
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Podemos monitorar o treinamento ao longo das épocas por meio do log gerado a cada época de treinamento; esses dados são salvos na variável history.

```
Epoch 1/500
19/19 [==============================] - 29s 492ms/step - loss: 1.1089 - box_output_loss: 0.1428 - class_output_loss: 0.9660 - box_output_mse: 0.1428 - clas
Epoch 2/500
19/19 [==============================] - 4s 215ms/step - loss: 0.4409 - box_output_loss: 0.0299 - class_output_loss: 0.4111 - box_output_mse: 0.0299 - class
Epoch 3/500
19/19 [==============================] - 3s 142ms/step - loss: 0.1847 - box_output_loss: 0.0154 - class_output_loss: 0.1693 - box_output_mse: 0.0154 - class
Epoch 4/500
```

Figura 16 - exemplo treinamento dataset
Fonte: autor

Etapa 8: Análise de Curvas de Perda e Acurácia.

Com o modelo treinado, podemos analisar as curvas de aprendizado do modelo para identificar problemas potenciais e garantir que o modelo aprendeu adequadamente.

```
import matplotlib.pyplot as plt
import pandas as pd
def plot_training_history(history):
    # Configuração do tamanho dos plots
    plt.figure(figsize=(18, 6))

    # Plotando o Loss de Treinamento e Validação
    plt.subplot(1, 3, 1)
    plt.plot(history['loss'], label='Loss de Treinamento')
    plt.plot(history['val_loss'], label='Loss de Validação')
    plt.title('Loss de Treinamento e Validação')
    plt.xlabel('Épocas')
    plt.ylabel('Loss')
```

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    plt.legend()

    # Plotando o Accuracy de Treinamento e Validação para a classificação
    plt.subplot(1, 3, 2)
    plt.plot(history['class_output_accuracy'],    label='Accuracy    de
Treinamento')
    plt.plot(history['val_class_output_accuracy'],  label='Accuracy  de
Validação')
    plt.title('Accuracy de Treinamento e Validação para Classificação')
    plt.xlabel('Épocas')
    plt.ylabel('Accuracy')
    plt.legend()

    # Plotando o MSE de Treinamento e Validação para a detecção
    plt.subplot(1, 3, 3)
    plt.plot(history['box_output_mse'], label='MSE de Treinamento')
    plt.plot(history['val_box_output_mse'], label='MSE de Validação')
    plt.title('MSE de Treinamento e Validação para Detecção')
    plt.xlabel('Épocas')
    plt.ylabel('MSE')
    plt.legend()

    plt.tight_layout()
    plt.show()

# Carrega o histórico
history_data = pd.DataFrame(history.history)
plot_training_history(history_data)
```

Código-fonte 20 – Exibe resultado
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

É importante realizar a avaliação gráfica de todas as métricas a fim de compreender de forma mais abrangente o treinamento da rede, conforme a imagem a seguir.

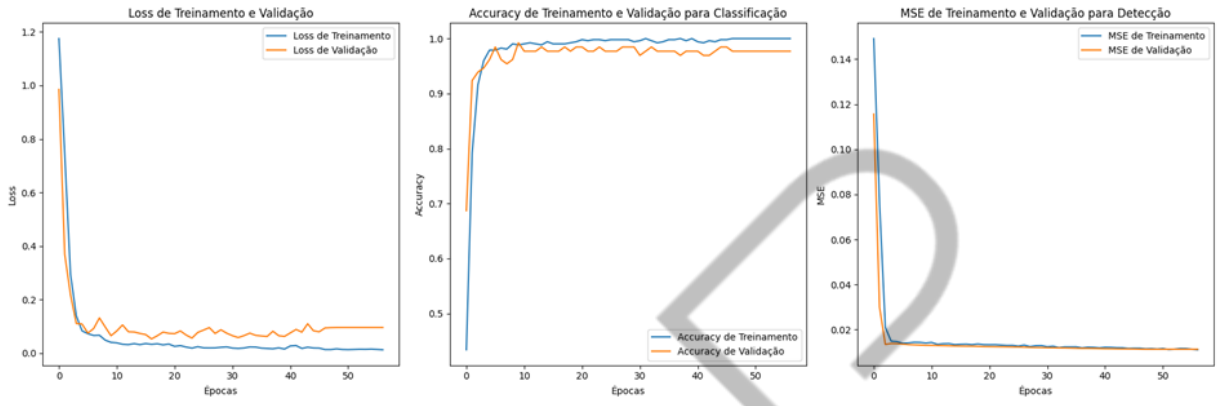


Figura 17 - Treinamento resultado
Fonte: autor

Terminamos o nosso treinamento e, para finalizar, precisamos salvar nosso modelo inteiro em um único arquivo e os nomes das classes.

```
import json

# Salva o modelo inteiro em um arquivo HDF5 .h5.
model.save(modelo.h5')

# Dicionário que mapeia índices para nomes de classes
class_names = {0: 'Borboleta', 1: 'Rosto de Puma', 2: 'Elefante'}

# Salva o dicionário de nomes de classes em um arquivo JSON
with open('class_names.json', 'w') as file:
    json.dump(class_names, file)

print("Modelo e Nomes das classes salvos com sucesso!")
```

Código-fonte 21 – Salva modelo
Fonte: autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Etapa 9: Usando o Modelo Treinado para Novas Imagens.

Com modelo treinado e validado, podemos utilizar em novas aplicações. Para demonstrar, vamos realizar predição em novas imagens, demonstrando como processar essas imagens e interpretar os resultados do modelo para detecção de objetos. Esse processo envolve várias etapas críticas para preparar as imagens para o modelo e, em seguida, aplicar o modelo para obter e interpretar os resultados.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import json
from tensorflow.keras.models import load_model

# Carrega o modelo
model = load_model('modelo.h5')

# Carrega os nomes das classes
with open('class_names.json', 'r') as file:
    label_names_dict = json.load(file)
    label_names = sorted(label_names_dict.values())

def preprocess(img, image_size=300):
    """Preprocessa a imagem para o formato adequado ao modelo."""
    image = cv2.resize(img, (image_size, image_size))
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image = image.astype("float32") / 255.0
    image = np.expand_dims(image, axis=0)
    return image
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
def postprocess(image, results):
    """Processa os resultados do modelo para obter rótulos e coordenadas
das caixas delimitadoras."""
    bounding_box, class_probs = results
    if class_probs.ndim > 1:
        class_probs = class_probs.flatten()
    class_index = np.argmax(class_probs)
    class_index >= len(label_names)
    class_label = label_names[class_index]
    h, w = image.shape[:2]
    x1, y1, x2, y2 = bounding_box[0]
    x1, x2 = int(x1 * w), int(x2 * w)
    y1, y2 = int(y1 * h), int(y2 * h)
    return class_label, (x1, y1, x2, y2), class_probs[class_index]

def predict(image, returnimage=False, scale=0.9):
    processed_image = preprocess(image)
    results = model.predict(processed_image)
    label, (x1, y1, x2, y2), confidence = postprocess(image, results)
    cv2.rectangle(image, (x1, y1), (x2, y2), (0, 255, 100), 2)
    cv2.putText(image, f'{label} ({confidence:.2f})', (x1, y2 + int(35 *
scale)), cv2.FONT_HERSHEY_COMPLEX, scale, (200, 55, 100), 2)
    if returnimage:
        return image
    else:
        plt.figure(figsize=(10, 10))
        plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
        plt.axis('off')
        plt.show()
```

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Exemplo de uso

image_path = 'elephant.jpg'  # Substitua pelo caminho correto conforme
necessário

image = cv2.imread(image_path)

predict(image, scale=1)
```

Código-fonte 22 – Testa resultado
Fonte: autor

Em resumo, neste capítulo, aprendemos:

- Técnicas de processamento de imagens Digitais;
- Utilização de visão computacional para detecção de pessoas;
- Aplicação de deep learning para classificação de objetos;
- Criação de uma arquitetura de detecção de objetos a partir de um modelo de classificação.

4 HANDS ON: PRATICANDO O USO DE TENSORFLOW

4.1 Aquecimento

Neste capítulo, vimos como algumas técnicas de deep learning são aplicadas em problemas de visão computacional como detecção de objetos e faces, vimos a importância do pré-processamento como normalização e data augmentation. Antes de começar o projeto do Capítulo, certifique-se de ter acesso ao Google Colab e que o ambiente está ativo para uso da GPU. Isso será particularmente útil na etapa de treinamento do modelo.

Dica: para alterar seu ambiente para contendo uma GPU no Google Colab, basta clicar no ambiente de execução, depois em “Alterar o tipo de ambiente de execução” e então selecionar “T4 GPU” como acelerador de hardware.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

4.2 Seu desafio

Sua tarefa é criar um modelo de detecção. Para isso, vamos utilizar o conjunto de dados **Caltech 101**, que é um conjunto de dados popular utilizado em pesquisas de visão computacional, particularmente em tarefas de reconhecimento de objetos. Ele contém cerca de 9.144 imagens distribuídas em 101 categorias de objetos, com algumas categorias adicionais de fundo. Cada categoria inclui aproximadamente 40 a 800 imagens.

As imagens no Caltech 101 são variadas e incluem objetos como animais, utensílios domésticos, instrumentos musicais, plantas, veículos, entre outros. A diversidade das imagens torna esse conjunto de dados adequado para treinar e testar algoritmos de classificação e detecção de objetos.

Você deve:

- O dataset é encontrado no link: https://data.caltech.edu/records/mzrjq-6wc02;
- Aplicar as técnicas de pré-processamento e aumento de dados;
- Definir a arquitetura usando o TensorFlow;
- Treinar o modelo usando GPU.

Espera-se uma Acurácia de pelo menos 0,85.

4.3 Dicas de solução

Para esse desafio, é esperado que você desenvolva um modelo de deep learning capaz de identificar e localizar os objetos presentes nas imagens fornecidas pelo dataset Caltech. A proposta é que você aplique os conceitos aprendidos sobre detecção de objetos de maneira prática, construindo um modelo robusto que demonstre sua capacidade de generalizar a partir de dados variados.

Para realizar esse trabalho, você deve criar um novo notebook no Google Colab e seguir as nove etapas citadas na seção "Do Classificador ao Detector". Aqui estão as etapas detalhadas para guiá-lo através do processo:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Visão computacional (Deteção de faces e objetos)

Etapa 1: baixe o Conjunto de Dados. Inicie o projeto adquirindo o dataset Caltech 101 e verifique se os dados foram importados corretamente;

Etapa 2: realize o data augmentation. Isso inclui alterações como rotação, escalonamento e mudanças na iluminação das imagens, ajudando a melhorar a robustez do modelo. Não esqueça de aplicar as transformações nos rótulos. Cuidado para não utilizar parâmetros como flp vertical, pois pode gerar imagens fora de contexto;

Etapa 3: pré-processamento, prepare os dados para o treinamento ajustando o tamanho das imagens, normalizando os valores dos pixels e convertendo as anotações para o formato adequado para treinamento;

Etapa 4: visualize os Dados com suas Anotações, é importante algumas amostras das imagens juntamente com suas anotações (labels) para entender como os dados estão estruturados e como as etiquetas estão sendo aplicadas. Não tem como prosseguir sem esta validação;

Etapa 5: divida os Dados em Conjuntos de Treino e Validação. Separe os dados em dois grupos: um para treinar o modelo e outro para validar seu desempenho durante o treinamento, garantindo que o modelo não esteja apenas decorando as imagens;

Etapa 6: construa o Modelo de Múltiplas Saídas, desenvolva um modelo que possa não apenas classificar, mas também determinar a localização dos objetos nas imagens. Isso geralmente envolve saídas múltiplas, uma para classificação e outra para a localização (bounding boxes). Teste com arquiteturas leves e performáticas como a **MobileNetV2**;

Etapa 7: compile e Treine o Modelo, é importante definir a função de perda e o otimizador. Acompanhe de perto a evolução do treinamento para garantir que o modelo está aprendendo adequadamente. Caso seja necessário, faça os ajustes no otimizador e função de perda e treine novamente;

Etapa 8: visualize o desempenho do modelo por meio das curvas de perda e acurácia. É fundamental entender se há necessidade de ajustes na arquitetura ou nos parâmetros de treinamento;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Etapa 9: teste Seu Modelo em Novas Imagens. Finalmente, avalie a capacidade do modelo de generalizar, aplicando-o a imagens novas, nunca vistas durante o treinamento, para verificar sua eficácia em situações reais.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

BHOWMIK, M. K. (2024). Fundamentals of Object Detection. Em Computer Vision (1ª ed., pp. 12). Chapman and Hall/CRC. Disponível em: <https://doi.org/10.1201/9781003432036-1>. Acesso em: 28 mai. 2024.

CALIN, Ovidiu. **Deep learning architectures**. New York City: Springer International Publishing, 2020.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep learning**. MIT press, 2016.

HAYKIN, Simon. **Redes neurais:** princípios e prática. Bookman Editora, 2008.

Karpathy, A., et al. (2017). **Lecture 11:** Detection and Segmentation. Stanford University. Disponível em <http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture11.pdf>. Acesso em: 14 abr. 2024.

LUGER; George F. **Inteligência Artificial**, 6ª ed. São Paulo: Pearson Education do Brasil, 2013 (biblioteca virtual).

MISHRA, Ranjan Kumar; REDDY, GY Sandesh; PATHAK, Himanshu. **The understanding of deep learning:** A comprehensive review. Mathematical Problems in Engineering, 2021, 2021: 1-15.

MITCHELL, Tom M. **Machine Learning**. 1st edition, New York, United States: McGraw-Hill Education, 1997.

RUSSEL, Stewart; NORVING, Peter. **Inteligência artificial**. 3ª. Ed., Rio de Janeiro: Campus, 2012.

Taha Anwar (2020). **Classification with localization**. Disponível em: <https://learnopencv.com/classification-with-localization/>. Acesso em: 14 abr. 2024.

WANI, M. Arif; KHOSHGOFTAAR, Taghi M.; PALADE, Vasile (ed.). **Deep Learning Applications**, Volume 3. Springer, 2021.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com